

Inception de l'idée

En pensant à différents jeux simples qu'on pourrait programmer sans connaître tous les racoins d'un langage, on s'est arrêté sur les animaux de compagnie virtuelle de type Tamagotchi.(Bandai, 2024) Ce type de jeux permet l'interaction de l'utilisateur avec un « animal » virtuel à travers le clic de boutons afin de garder l'animal en vie le plus longtemps possible alors que l'animal se rapproche de la « mort » avec le passage du temps. Le progrès de l'utilisateur est indiqué par l'âge de l'animal, qui augmente avec le passage du temps dans le jeu. Afin de se faire une idée de comment ce genre de jeux pouvait être bâti en python, on a demandé à ChatGPT de bâtir deux exemples de jeux de style Tamagotchi, dont un utilisant une interface graphique afin de faire jouer l'utilisateur au jeu et l'autre utilisant la console pour communiquer avec l'utilisateur. (OpenAI, 2024) Ces exemples ont permis de concrétiser les idées qu'on avait quant à ce qu'on voulait inclure à notre jeu et comment on voulait le présenter et implémenté ces idées.

Spécifiquement, on a décidé d'implémenter les attributs de faim, d'énergie et de joie sur un total de 100 points qui, si sa valeur atteint 0 dans le cas de l'énergie et de la joie ou 100 dans le cas de la faim, entraine la mort de l'animal et la fin du jeu. Ces attributs diminuent ou augmentent, respectivement, avec le passage du temps dans le jeu et le seul moyen de renverser cet effet et d'interagir avec l'animal à travers des boutons qui permettent de prendre soin de l'animal. Les boutons choisis permettent de nourrir l'animal, de le faire dormir, de lui faire prendre un bain, de jouer avec lui et de l'amener chez le vétérinaire. Cette dernière activité est particulièrement nécessaire puisqu'on a décidé d'implémenter la possibilité que l'animal tombe malade aléatoirement, ce qui lui cause de dépenser ses points d'attributs, donc de se rapprocher de la mort, plus rapidement. Finalement, on a aussi décidé d'implémenter la possibilité de flatter l'animal, ce qui a un effet positif sur sa joie sans avoir à cliquer de bouton en tant que tel.

Pour ce qui est de l'apparence du jeu, on s'en est tenue aux formes et couleurs disponibles à travers le module pygame.(Pygame Community et al., 2024) On a décidé de donner différentes couleurs à chaque bouton et donné au jeu un aspect

similaire au mode sombre des logiciels modernes. On a décidé d'indiquer à l'utilisateur la valeur des attributs de l'animal en tout temps avec des barres qui se remplissent plus la valeur de l'attribut est grande en plus d'écrire la valeur numérique du nombre de points sur le maximum de 100 que l'animal possède pour cet attribut. On a aussi décidé de changer la couleur des barres d'attributs au rouge lorsque ceux-ci se retrouvent près de la mort.

Implémentation des fonctions

Afin de programmer les fonctions nécessaires à l'implémentation des idées présentées ci-haut, on a décidé de commencer par tout programmer dans un seul fichier. On a commencé avec un programme très simple qui ne faisait qu'afficher l'animal dans la fenêtre, et on a ajouté au fur et à mesure différentes fonctionnalités en les testant rigoureusement en les affichant dans la fenêtre de jeux avant de passer à la prochaine. Cette approche est considérablement plus efficace que l'approche qu'on utilise d'habitude de tout diviser et tout programmer avant de passer plusieurs heures à tester et déboguer. Malgré qu'on se retrouve à passer beaucoup moins de temps à déboguer et tester le code, la méthode utilisée n'est pas toujours possible puisqu'elle nécessite l'accès complet au code et aux composantes graphiques et ne fonctionne que pour des applications à petite échelle.

Comme mentionner plus haut, on a commencé par créer la classe `Pet` et lui implémenter la fonction `draw(window, x, y)` qui permet de dessiner l'animal dans la fenêtre `window` aux coordonnées `x` et `y` passées en argument. On a testé le succès de cette fonction, et de toutes les suivantes, dans l'implémentation d'une boucle infinie à laquelle on ajoutera les fonctions nécessaires à l'affichage au fur et à mesure qu'elles seront créées.

Suite au succès de cette fonction, on a implémenté la fonction `draw_buttons(window)` à la classe `Buttons` afin de pouvoir, en un seul appel, dessiner tous les boutons nécessaire au jeu dans la fenêtre `window`. On a ajouté cette fonction à la boucle infinie afin de la tester, démontrant son succès.

Ensuite on s'est penché sur les barres d'attributs. Leur implémentation nécessitait l'implémentation des attributs dans la classe `Pet` afin de pouvoir suivre

leurs valeurs à travers le logiciel. On a fini par ajouter la fonction `draw_stats(window)` qui permettait d'afficher les barres de tous les attributs empilées au haut de la fenêtre de jeux `window`, chacun avec son texte unique reflétant la valeur courante de l'attribut sur 100 ainsi qu'avec la coloration verte (ou rouge) de la barre reflétant cette fraction. La coloration de la barre était en effet programmée pour changer selon si l'attribut en question était à 10 points ou moins de causer la mort afin d'alerter le joueur de la mort imminente de l'animal.

Enfin, on a pu se pencher sur l'implémentation des clics des boutons. On a utilisé la fonction `touche(x, y)`, inspirée de celle présente dans le livre du cours et reprenant son implémentation dans la boucle infinie faisant rouler le jeu.(Godin & Lemire, 2024) La fonction vérifiait si la position `(x, y)` passée en arguments se trouvait à l'intérieur d'un des boutons ou à l'intérieur du rectangle de l'animal. Si la position se trouvait dans l'une de ces zones, on appelait la fonction d'interaction avec l'animal correspondante. Ces fonctions ont été ajoutées à la classe `Pet` spécifiquement pour rendre les interactions avec l'animal plus facile à déclencher à partir des autres modules. Afin de vérifier si la position du clic se trouvait dans la zone d'un bouton ou de dessin de l'animal, on a profité de la méthode `collidepoint(x, y)` que les `pygame.Rectangle` possèdent qui retourne `True` si le point passé en argument se trouve dans le rectangle et `False` sinon.

Maintenant qu'on pouvait contrer les effets du temps, on a développé la fonction qui allait être utilisée pour simuler l'effet du passage du temps sur l'animal. Puisque la fonction `time_passes()` serait appelée à chaque passage dans la boucle infinie roulant le jeu, soit 60 fois par seconde, tel que précisé quand on a configuré l'horloge afin d'assurer une animation fluide, on a décidé de modifier les attributs de l'animal par une très petite valeur à chaque appel de la fonction. De plus, la fonction modifie la valeur de l'âge de l'animal afin que celui-ci vieillisse d'un total d'un an par minute de jeux. Finalement, la fonction vérifie qu'elle n'a pas tué l'animal, et envoie une erreur, qui sera expliquée dans la section suivante, si c'est le cas. Puisque l'animal pouvait maintenant mourir, on en a

profité pour implémenter l'écran de fin de jeu qui serait affiché si la boucle de jeu recevait l'exception lancée par la mort.

Maintenant que les fonctions nécessaires à jouer au jeu étaient implémentées, on s'était concentré sur la sauvegarde et le chargement du jeu. On s'était principalement inspiré des jeux que ChatGPT avait générés au début de l'exercice afin d'utiliser un fichier JSON identifier par le nom de l'animal pour enregistrer la valeur de tous ses attributs. La fonction `save()` ne faisait qu'enregistrer les informations courantes de l'animal dans un fichier JSON contenant le nom du pet, remplaçant les sauvegardes précédentes s'il y en a, puis affichait un message dans la console confirmant la sauvegarde. La fonction `load()` a été identifiée comme une méthode de classe afin de pouvoir être appelée sans créer de `Pet` puisque celle-ci créerait un `Pet` si les données d'un animal étaient trouvées. Si le nom entré dans la console ne correspondait à aucun animal avec un fichier JSON existant, elle attrapait l'erreur générée et retournait `None` et le programme créait un nouvel animal avec ce nom afin de commencer le jeux.

On s'était ensuite penché sur les parties du jeu complémentaire, mais non essentielle. Tout d'abord, on a implémenté la possibilité à l'animal de se déplacer aléatoirement vers la gauche ou la droite de 25 pixels à tous les tics de l'horloge. Ceci donne à l'animal un semblant de vie. Ensuite, on a implémenté la possibilité que l'animal tombe malade. Ceci avait 0.5% de chance de se produire par seconde de jeu. Si l'animal tombait malade, le passage du temps avait maintenant un effet plus nocif pour lui. Le seul moyen de ramener un animal à la santé était de l'amener chez le vétérinaire, ce à quoi servait un des boutons. Afin d'afficher à l'utilisateur que son animal était malade ou qu'il était à dix points de mourir, on a ajouté une phrase indicatrice sur l'écran qui dirait à l'utilisateur laquelle des situations était en train de se produire afin qu'il puisse la rectifier, et on a aussi ajouté la possibilité de dessiner l'animal avec une moue plutôt qu'un sourire s'il était entrain de subir un de ces effets.

Implémentation des exceptions et leur utilité

Le logiciel utilise deux gestions d'exception qui illustre comment celles-ci peuvent être utilisées. Cependant, le fait que le logiciel ne prévoit pas de gérer d'autre exception est, en soit, une manière de gérer les exceptions. En effet, le fait de ne pas permettre à l'utilisateur de généré des erreurs par leurs mauvaises entrées est en tant que tel un manière de géré et de minimisé les erreurs. Puisque toutes les entrées des utilisateurs dans le jeu sont par l'intermission des boutons, ceci permet au logiciel de gérer tous les potentiels conflits à l'intérieur de celle-ci sans permettre à l'utilisateur de commettre des erreurs d'entrées ou de commandes qui interrompraient l'exécution de l'application. Le seul cas où l'utilisateur est libre d'entrer des données est lors du chargement de son animal dans le jeu.

Si l'utilisateur entre un nom dont le fichier de sauvegarde n'existe pas dans le répertoire courant, la fonction `open()` va lancer l'exception

`FileNotFoundError`. Cette exception sera attrapée dans la fonction `load(name)` elle-même et ne fera que changer la valeur de retour afin que le logiciel soit notifié de l'absence de fichier de sauvegarde afin qu'il crée un nouvel animal. Ceci prévient l'arrêt du logiciel tout en permettant de combiner en un seul interface utilisateur la création d'animaux et le chargement d'animal existant.

Finalement, si l'animal se retrouve à perdre tous ses points d'attributs (ou à tous les gagner dans le cas de la faim), le logiciel va utiliser l'exception `Death(cause)` afin de notifier le jeu de la mort. Cette exception doit être appelée en précisant la cause de la mort, puisque cette information permet de personnaliser le message de mort affiché sur l'écran de fin de jeu. On remarque que seule la fonction `time_passes()` la lance, puisque c'est le seul endroit dans le logiciel qui peut causer la mort de l'animal. Cette fonction est appelée uniquement dans la boucle de jeu, donc ces celle-ci qui contient de `try except else` qui l'attend. Si l'exception de la mort est évitée, le programme peut donc continuer en exécutant le bloc `else`, mais si la mort se produit il exécute le bloc `except` qui ne permet à l'utilisateur qu'une action, celle de fermer le jeu.

Bibliographie

Bandai. (2024). *Official Tamagotchi Site*. Official Tamagotchi Site.

<https://tamagotchi-official.com/us/>

Godin, R., & Lemire, D. (2024). *Programmation avec Python : Des jeux au Web*.

OpenAI. (2024). *ChatGPT* (Version ChatGPT May 16, 2024 Version) [Python].

OpenAI. <https://chatgpt.com>

Pygame Community, Andrews, A., Theden, L., Lönnhager, D., Barbalho, T. J.,

Dudfield, R., Dudfield, N., & Kluyver, T. (2024). *Pygame* (Version 2.6.1)

[Python, C, Ruby]. <https://www.pygame.org/>

Annexes

Annexe 1 – fichier Main.py

```
import pygame
import random
import Pet

pygame.init()
LARGUEUR_FENETRE = 1200
HAUTEUR_FENETRE = 1000
fenetre = pygame.display.set_mode((LARGUEUR_FENETRE,
HAUTEUR_FENETRE))
pygame.display.set_caption('Virtual Pet')
FONT=pygame.font.Font(None, 32)
horloge = pygame.time.Clock()

class Death(Exception):
    """exception to call if pet dies

    Args:
        cause (str): cause of death
    """
    def __init__(self, cause):
        self.cause=cause

def main() :

    #essaye de loader un pet existant sinon en cree un
    nouveau
    pet_name = input("What is your pet's name? ")
    pet1 = Pet.Pet.load(pet_name)
    if pet1==None : pet1=Pet.Pet(pet_name)

    fin = False
    while not fin :
        event = pygame.event.poll()
        if event.type == pygame.QUIT :
            fin = True #ferme le jeux quand on quitte
        else :
            #verifie si un bouton est clique, applique
            son effet
            if event.type == pygame.MOUSEBUTTONUP :
                pet1.buttons.touche(event.pos[0], event.pos[1])
                try :
                    #applique le passage du temp
```

```
        pet1.time_passes()
        fenetre.fill(pygame.Color('grey10'))
    except Death as death : #si le pet est mort,
on indique la fin du jeu
        pygame.draw.rect(fenetre,
pygame.Color('red'), pet1.rect)
        text = FONT.render(pet1.name+" died of
"+death.cause+"!! Please close the game :(", True,
pygame.Color('red'))
        fenetre.blit(text, [200, 500])
    else : #si le pet est encore en vie, on
redessine tout!
        #deplacement aleatoire du pet, 10% de
chance de se deplacer de max 25 pixel a gauche ou a
droite a tous les ticks
        if random.randrange(0, 10, 1)==1 :
            new_x =
pet1.rect.x+random.randrange(-25,25,1)
            if new_x<LARGUEUR_FENETRE-
pet1.LARGEUR_PET and new_x>=0 : pet1.rect.x=new_x
            #0.5% de chance de tomber malade par sec
            if random.randrange(0, 1200, 1)==1 :
pet1.is_sick=True
            #dessine les objets dans la fenetre
pet1.draw(fenetre, pet1.rect.x,
pet1.rect.y)
            pet1.draw_stats(fenetre)
            pet1.draw_effects(fenetre)
            pet1.buttons.draw_buttons(fenetre)
            #met a jour le display
pygame.display.flip()
            horloge.tick(60) #60 ticks/frames par seconde

pygame.quit()

if __name__ == "__main__":
    main()
```

Annexe 2 – fichier Pet.py

```
import pygame
import json
import math
import Main
import Buttons
```

```
class Pet :
    """The Virtual Pet

    Args:
        LARGEUR_PET : dimension du pet en x
        HAUTEUR_PET : dimension du pet en y
        name : nom du pet
        rect (Rect) : rectangle dans laquelle le pet est
dessine
    Stats :
        hunger : niveau de faim, min 0, max 100
        energy : niveau d'energie, min 0, max 100
        happiness : niveau de joie, min 0, max 100
        buttons : les boutons pour agir sur le pet
        age : age du pet, min 0
        is_dying : si le pet est sur le point de mourir,
boolean
        is_sick : si le pet est malade, boolean

    """
    def __init__(self, name, x=Main.LARGUEUR_FENETRE/2,
y=Main.HAUTEUR_FENETRE/2, hunger=50, energy=50,
happiness=50, age=0, is_dying=False, is_sick=False):
        if (age<18) :
            self.LARGEUR_PET = 100
            self.HAUTEUR_PET = 150
        else :
            self.LARGEUR_PET = 140
            self.HAUTEUR_PET = 210
        self.name = name
        self.rect = pygame.Rect(x, y, self.LARGEUR_PET,
self.HAUTEUR_PET)
        self.hunger = hunger
        self.energy = energy
        self.happiness = happiness
        self.Stats = [self.hunger, self.energy,
self.happiness]
        self.buttons = Buttons.Buttons(self)
        self.age = age
        self.is_dying=is_dying
        self.is_sick=is_sick

    def draw(self, window, x, y):
        """dessiner le pet dans la fenetre window au
coordonnees x,y
```

```

    Args:
        window (Surface): la fenetre ou dessiner le
pet
        x (int): coordonnee x d'ou dessiner le pet
        y (int): coordonnee y d'ou dessiner le pet
    """
    if (self.age<18) :
        self.LARGEUR_PET = 100
        self.HAUTEUR_PET = 150
    else :
        self.LARGEUR_PET = 140
        self.HAUTEUR_PET = 210
    self.rect = pygame.Rect(x, y, self.LARGEUR_PET,
self.HAUTEUR_PET)
    #HEAD
    pygame.draw.circle(window,
pygame.Color('lavender'), (x+self.LARGEUR_PET/2,y+self.HAU
TEUR_PET/6), (self.LARGEUR_PET/4))
    pygame.draw.circle(window, pygame.Color('black'),
(x+self.LARGEUR_PET/4+self.LARGEUR_PET/2/3,y+self.HAUTEUR
_PET/3/3),2)
    pygame.draw.circle(window, pygame.Color('black'),
(x+self.LARGEUR_PET/4+self.LARGEUR_PET/2*2/3,y+self.HAUTE
UR_PET/3/3),2)
    #si mourant ou malade = frown, sinon smile
    if self.is_dying or self.is_sick :
        pygame.draw.arc(window,
pygame.Color('black'),
((x+self.LARGEUR_PET/4+self.LARGEUR_PET/2/4,y+self.HAUTEU
R_PET/3*2/3), (self.LARGEUR_PET/2/2, self.HAUTEUR_PET/3/2)), -
5*math.pi/6, 5*math.pi/6, 2)
    else :
        pygame.draw.arc(window,
pygame.Color('black'),
((x+self.LARGEUR_PET/4+self.LARGEUR_PET/2/4,y+self.HAUTEU
R_PET/3/4), (self.LARGEUR_PET/2/2, self.HAUTEUR_PET/3/2)), -
5*math.pi/6, -math.pi/6, 2)
    #BODY
    pygame.draw.polygon(window,
pygame.Color('skyblue'), ((x,y+self.HAUTEUR_PET/3), (x+self
.LARGEUR_PET,
y+self.HAUTEUR_PET/3), (x+self.LARGEUR_PET/2,
y+self.HAUTEUR_PET*2/3)))
    pygame.draw.polygon(window,
pygame.Color('skyblue'), ((x,y+self.HAUTEUR_PET), (x+self.L

```

```
ARGEUR_PET, y+self.HAUTEUR_PET),(x+self.LARGEUR_PET/2,
y+self.HAUTEUR_PET*2/3)))

    def draw_stats(self, window):
        """dessine les statistiques du pet dans la
fenetre window

        Args:
            window (Surface): fenetre ou le pet est
dessine
        """
        #write out stats
        hunger_text = "Hunger :
"+str(round(self.hunger,2))+"/100"
        hunger_render = Main.FONT.render(hunger_text,
True, pygame.Color('black'))
        energy_text = "Energy :
"+str(round(self.energy,2))+"/100"
        energy_render = Main.FONT.render(energy_text,
True, pygame.Color('black'))
        happiness_text = "Happiness :
"+str(round(self.happiness,2))+"/100"
        happiness_render =
Main.FONT.render(happiness_text, True,
pygame.Color('black'))
        #draw bars and stat text
        Stats_Text = [[hunger_text, hunger_render,
self.hunger], [energy_text, energy_render, self.energy],
[happiness_text, happiness_render, self.happiness]]
        for i,stat_text in enumerate(Stats_Text):
            #dessine la bar vide
            pygame.draw.rect(window,
pygame.Color('grey30'), pygame.Rect(50, 50+i*(45),
Main.LARGUEUR_FENETRE-100, 40))
            #met en rouge la barre si on est a 10 points
de la mort
            if (("Hunger" not in stat_text[0]) and
stat_text[2]<=10) or (("Hunger" in stat_text[0]) and
stat_text[2]>=90):
                pygame.draw.rect(window,
pygame.Color('red'), pygame.Rect(50, 50+i*(45),
(Main.LARGUEUR_FENETRE-100)*stat_text[2]/100, 40))
            else : pygame.draw.rect(window,
pygame.Color('lightgreen'), pygame.Rect(50, 50+i*(45),
(Main.LARGUEUR_FENETRE-100)*stat_text[2]/100, 40))
            window.blit(stat_text[1], (55, 55+i*(45)))
```

```
def draw_effects(self, window):
    """ecris l'age et si le pet est malade ou mourant
    sous les stats

    Args:
        window (Surface): fenetre ou le pet est
dessine
    """
    #ecris et affiche l'age a droit au dessus des
effets
    age_text = "Age : "+str(math.floor(self.age))
    age_render = Main.FONT.render(age_text, True,
pygame.Color('white'))
    window.blit(age_render, (Main.LARGUEUR_FENETRE-
300, 205))
    #ecris et affiche si malade/mourant a droite en
dessous de l'age
    if self.is_dying : effect_text=self.name+" is
dying!!"
    elif self.is_sick : effect_text=self.name+" is
sick!"
    else : effect_text=self.name+" is healthy"
    if "healthy" in effect_text :
text_color=pygame.Color('lightgreen')
    else : text_color=pygame.Color('red')
    effect_render = Main.FONT.render(effect_text,
True, text_color)
    window.blit(effect_render,
(Main.LARGUEUR_FENETRE-300, 250))

    ###actions qu'on peut appliquer sur le pet###
def feed(self) :
    self.hunger = max(0, self.hunger-10)

def play(self) :
    self.happiness = min(100, self.happiness+10)
    self.energy= max(0, self.energy-10)

def sleep(self) :
    self.energy = min(100, self.energy+20)
    self.age += 1

def bathe(self) :
    self.happiness = min(100, self.happiness+15)
```

```
def visit_vet(self) :
    self.hunger = max(0, self.hunger-5)
    self.energy = min(100, self.energy+5)
    self.is_sick = False

def pet(self) :
    self.happiness = min(100, self.happiness+5)
    ###

def time_passes(self) :
    """effet du temps sur les stats du pet, pire s'il
est malade
    """
    if self.is_sick :
        self.hunger = min(100, self.hunger+0.003)
        self.energy = max(0, self.energy-0.005)
        self.happiness = max(0, self.happiness-0.003)
    else :
        self.hunger = min(100, self.hunger+0.002)
        self.energy = max(0, self.energy-0.002)
        self.happiness = max(0, self.happiness-0.001)
    self.age += (1/60/60) #+1 age a chaque minute
    #verifie que le pet n'est pas mort
    if self.hunger>=100 : raise
Main.Death("starvation")
    elif self.energy<=0 : raise
Main.Death("tiredness")
    elif self.happiness<=0 : raise
Main.Death("boredom")
    #verifie que le pet n'est pas mourant
    elif self.hunger>=90 or self.energy<=10 or
self.happiness<=10 : self.is_dying=True
    else : self.is_dying=False

    ###methodes load et save pour enregistrer les donnees
du pet entre les sessions de jeux###
    @classmethod
    def load(cls, name):
        try:
            with open(f"{name}_save.json", "r") as file:
                data = json.load(file)
                print("Game loaded!")
                return cls(**data)
        except FileNotFoundError:
            print("Save file not found. Creating a new
pet.")
```

```
        return None

    def save(self):
        data = {
            'name': self.name,
            'hunger': self.hunger,
            'happiness': self.happiness,
            'energy': self.energy,
            'age': self.age,
            'is_dying': self.is_dying,
            'is_sick' : self.is_sick
        }
        with open(f"{self.name}_save.json", "w") as file:
            json.dump(data, file)
        print("Game saved!")
    ###
```

Annexe 3 – fichier Buttons.py

```
import pygame
import Main

class Buttons :
    """classe specifique pour les boutons d'actions
    Args :
        BUTTON_SIZE : la taille de tous les boutons
        BUTTON_Y : HAUTEUR A LAQUELLE METTRE LES BOUTONS
        pet : le pet affecter par les actions des boutons
        feed_button_zone : rectangle dans laquelle se
trouve le bouton "Feed"
        play_button_zone : rectangle dans laquelle se
trouve le bouton "Play"
        sleep_button_zone : rectangle dans laquelle se
trouve le bouton "Sleep"
        bathe_button_zone : rectangle dans laquelle se
trouve le bouton "Bathe"
        visit_vet_button_zone : rectangle dans laquelle
se trouve le bouton "Visit Vet"
        save_button_zone : rectangle dans laquelle se
trouve le bouton "Save Game"
    """
    def __init__(self, pet):
        self.BUTTON_SIZE= [200, 50]
        self.BUTTON_Y = Main.HAUTEUR_FENETRE-100
        self.pet=pet
```

```
        self.feed_button_zone = pygame.Rect(25,
self.BUTTON_Y, self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])
        self.play_button_zone = pygame.Rect(275,
self.BUTTON_Y, self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])
        self.sleep_button_zone = pygame.Rect(500,
self.BUTTON_Y, self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])
        self.bathe_button_zone = pygame.Rect(725,
self.BUTTON_Y, self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])
        self.visit_vet_button_zone = pygame.Rect(950,
self.BUTTON_Y, self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])
        self.save_button_zone = pygame.Rect(50, 205,
self.BUTTON_SIZE[0], self.BUTTON_SIZE[1])

    def draw_buttons(self, window):
        """dessine les boutons pour interagir avec le pet

        Args:
            window (Surface): fenetre dans laquelle se
trouve le pet
        """
        #FEED BUTTON
        pygame.draw.rect(window,
pygame.Color('greenyellow'), self.feed_button_zone)
        feed_text=Main.FONT.render('Feed', True,
pygame.Color('black'))
        window.blit(feed_text,
(self.feed_button_zone.x+75, self.feed_button_zone.y+15))
        #PLAY BUTTON
        pygame.draw.rect(window, pygame.Color('orange'),
self.play_button_zone)
        play_text=Main.FONT.render('Play', True,
pygame.Color('black'))
        window.blit(play_text,
(self.play_button_zone.x+75, self.play_button_zone.y+15))
        #SLEEP BUTTON
        pygame.draw.rect(window, pygame.Color('navy'),
self.sleep_button_zone)
        sleep_text=Main.FONT.render('Sleep', True,
pygame.Color('white'))
        window.blit(sleep_text,
(self.sleep_button_zone.x+75,
self.sleep_button_zone.y+15))
        #BATHE BUTTON
        pygame.draw.rect(window,
pygame.Color('turquoise1'), self.bathe_button_zone)
```

```
        bathe_text=Main.FONT.render('Bathe', True,
pygame.Color('black'))
        window.blit(bathe_text,
(self.bathe_button_zone.x+75,
self.bathe_button_zone.y+15))
        #VISIT_VET BUTTON
        pygame.draw.rect(window,
pygame.Color('firebrick'), self.visit_vet_button_zone)
        visit_vet_text=Main.FONT.render('Visit Vet',
True, pygame.Color('white'))
        window.blit(visit_vet_text,
(self.visit_vet_button_zone.x+50,
self.visit_vet_button_zone.y+15))
        #SAVE BUTTON
        pygame.draw.rect(window,
pygame.Color('darkgreen'), self.save_button_zone)
        save_text=Main.FONT.render('Save Game', True,
pygame.Color('white'))
        window.blit(save_text,
(self.save_button_zone.x+50, self.save_button_zone.y+15))

    def touche(self, x, y) :
        """verifie si un bouton (ou le pet) est toucher,
puis applique l'effet du bouton sur le pet

        Args:
            x (int): coordonnee x de la souris au moment
du clic
            y (int): coordonnee y de la souris au moment
du clic
        """
        worked = True
        if self.feed_button_zone.collidepoint(x,y) :
self.pet.feed()
        elif self.play_button_zone.collidepoint(x,y) :
self.pet.play()
        elif self.sleep_button_zone.collidepoint(x,y) :
self.pet.sleep()
        elif self.bathe_button_zone.collidepoint(x,y) :
self.pet.bathe()
        elif self.visit_vet_button_zone.collidepoint(x,y)
: self.pet.visit_vet()
        elif self.pet.rect.collidepoint(x,y) :
self.pet.pet()
        elif self.save_button_zone.collidepoint(x,y) :
self.pet.save()
```

```
else : worked=False  
return worked
```